



提纲

4.1 自顶向下的分析

4.2 预测分析法

4.3 自底向上的分析

4.4 LR分析法

4.5 语法分析器自动生成工具

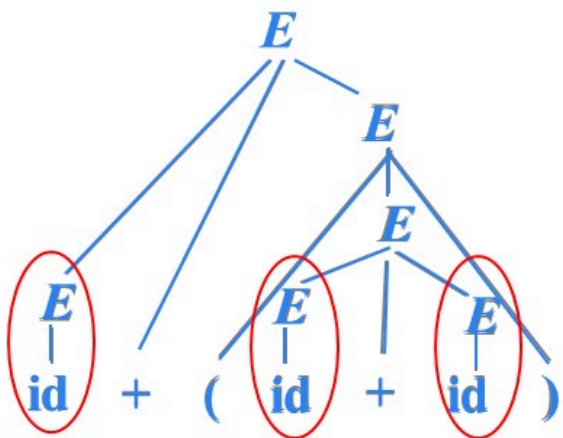
4.3 自底向上的语法分析

- 从分析树的底部(叶节点)向顶部(根节点)方向构造分析树
- 可以看成是将输入串 w 归约为文法开始符号 S 的过程
- 自顶向下的语法分析采用最左推导方式 (构造句子的最左推导)
自底向上的语法分析采用最左归约方式 (反向构造句子的最右推导)
- 自底向上语法分析的通用框架
 - 移入-归约分析(*Shift-Reduce Parsing*)

例：移入-归约分析

文法

- ① $E \rightarrow E+E$
- ② $E \rightarrow E * E$
- ③ $E \rightarrow (E)$
- ④ $E \rightarrow id$



最左归约

栈

\$
\$ id
\$ E
\$ E+
\$ E+(
\$ E+(id
\$ E+(E
\$ E+(E+
\$ E+(E+id
\$ E+(E+id
\$ E+(E+E
\$ E+(E
\$ E+(E)
\$ E+E
\$ E

剩余输入

id+(id+id) \$
+(id+id) \$
+(id+id) \$
(id+id) \$
id+id) \$
+id) \$
+id) \$
id) \$
) \$
) \$
) \$
\$
\$
\$

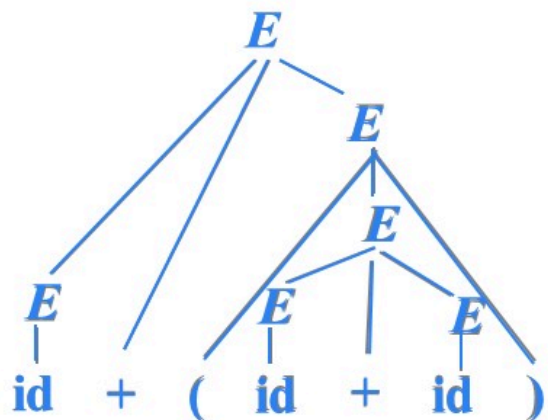
动作

移入
归约: $E \rightarrow id$
移入
移入
移入
归约: $E \rightarrow id$
移入
移入
归约: $E \rightarrow id$
归约: $E \rightarrow E+E$
移入
归约: $E \rightarrow (E)$
归约: $E \rightarrow E+E$

例：移入-归约分析

文法

- ① $E \rightarrow E+E$
- ② $E \rightarrow E * E$
- ③ $E \rightarrow (E)$
- ④ $E \rightarrow id$



最左归约

栈

\$
\$ id
\$ E
\$ E+
\$ E+(
\$ E+(id
\$ E+(E
\$ E+(E+
\$ E+(E+id
\$ E+(E+E
\$ E+(E
\$ E+(E)
\$ E+E
\$ E

剩余输入

id+(id+id) \$
+(id+id) \$
+(id+id) \$
(id+id) \$
id+id) \$
+id) \$
+id) \$
id) \$
) \$
) \$
) \$
\$
\$
\$

动作

移入
归约: $E \rightarrow id$
移入
移入
移入
归约: $E \rightarrow id$
移入
移入
归约: $E \rightarrow id$
归约: $E \rightarrow E+E$
移入
归约: $E \rightarrow (E)$
归约: $E \rightarrow E+E$

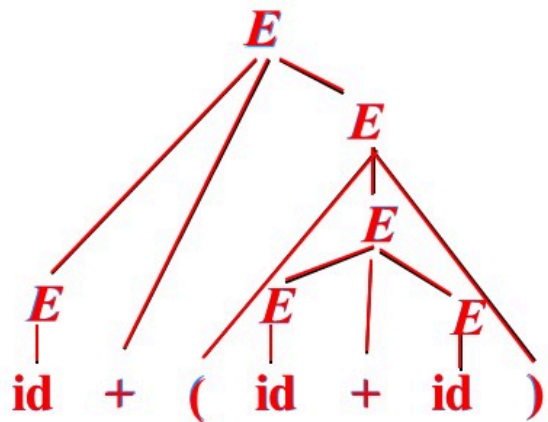
最右推导

例：移入-归约分析

栈内符号串 + 剩余输入 = “规范句型”

文法

- ① $E \rightarrow E+E$
- ② $E \rightarrow E * E$
- ③ $E \rightarrow (E)$
- ④ $E \rightarrow id$



最左归约

栈	剩余输入	动作
\$	id+(id+id) \$	
\$ id	+(id+id) \$	移入
\$ E	+(id+id) \$	归约: $E \rightarrow id$
\$ E +	(id+id) \$	移入
\$ E + (	id+id) \$	移入
\$ E + (id	+id) \$	移入
\$ E + (E	+id) \$	归约: $E \rightarrow id$
\$ E + (E +	id) \$	移入
\$ E + (E + id	) \$	移入
\$ E + (E + E	) \$	归约: $E \rightarrow id$
\$ E + (E	) \$	归约: $E \rightarrow E + E$
\$ E + (E)	\$	移入
\$ E + E	\$	归约: $E \rightarrow (E)$
\$ E	\$	归约: $E \rightarrow E + E$

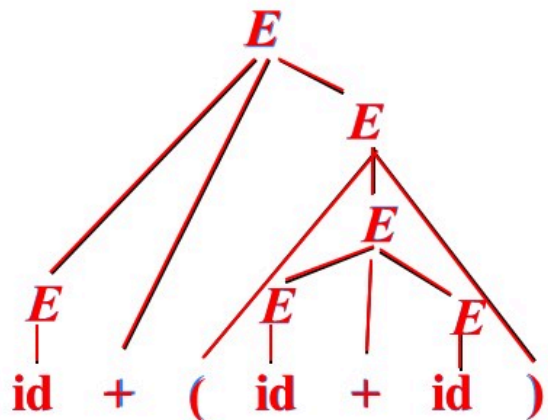
最右推导

例：移入-归约分析

栈内符号串 + 剩余输入 = “规范句型”

文法

- ① $E \rightarrow E+E$
- ② $E \rightarrow E * E$
- ③ $E \rightarrow (E)$
- ④ $E \rightarrow id$



最左归约

栈	剩余输入	动作
\$	id+(id+id) \$	
\$ id	+(id+id) \$	移入
\$ E	+(id+id) \$	归约: $E \rightarrow id$
\$ E +	(id+id) \$	移入
\$ E + (	id+id) \$	移入
\$ E + (id	+id) \$	移入
\$ E + (E	+id) \$	归约: $E \rightarrow id$
\$ E + (E +	id) \$	移入
\$ E + (E + id	) \$	移入
\$ E + (E + E	) \$	归约: $E \rightarrow id$
\$ E + (E + E +	) \$	归约: $E \rightarrow E + E$
\$ E + (E + E +)	\$	移入
\$ E + (E + E + E	\$	归约: $E \rightarrow (E)$
\$ E + (E + E + E +	\$	归约: $E \rightarrow E + E$

最右推导

移入-归约分析的工作过程

- 在对输入串的一次从左到右扫描过程中，语法分析器将零个或多个输入符号移入到栈的顶端，直到它可以对栈顶的一个文法符号串 β 进行归约为止
- 然后，它将 β 归约为某个产生式的左部
- 语法分析器不断地重复这个循环，直到它检测到一个语法错误，或者栈中包含了开始符号且输入缓冲区为空(当进入这样的格局时，语法分析器停止运行，并宣称成功完成了语法分析)为止

移入-归约分析器可采取的4种动作

- **移入**：将下一个输入符号移到栈的顶端
- **归约**：被归约的符号串的**右端**必然处于栈顶。语法分析器在栈中确定这个串的**左端**，并决定用哪个非终结符来替换这个串
- **接收**：宣布语法分析过程成功完成
- **报错**：发现一个语法错误，并调用错误恢复子例程

移入-归约分析中存在的问题

- 归约-归约冲突
- 移入-归约冲突

归约-归约冲突

例：

(1) $\langle S \rangle \rightarrow \text{var } \langle IDS \rangle : \langle T \rangle$

(2) $\langle IDS \rangle \rightarrow i$

(3) $\langle IDS \rangle \rightarrow \langle IDS \rangle , i$

(4) $\langle T \rangle \rightarrow \text{real} \mid \text{int}$

栈

剩余输入

动作

\$	var $i_A, i_B : \text{real}$	\$
\$ var	$i_A, i_B : \text{real}$	\$
\$ var i_A	, $i_B : \text{real}$	\$
\$ var $\langle IDS \rangle$	, $i_B : \text{real}$	\$
\$ var $\langle IDS \rangle ,$	$i_B : \text{real}$	\$
\$ var <u>$\langle IDS \rangle , i_B$</u>	: real	\$
\$ var $\langle IDS \rangle , \langle IDS \rangle$	: real	\$
\$ var $\langle IDS \rangle , \langle IDS \rangle :$	real	\$
\$ var $\langle IDS \rangle , \langle IDS \rangle :$	real	\$
\$ var $\langle IDS \rangle , \langle IDS \rangle : \langle T \rangle$		\$

移入
移入
归约
移入
移入
归约
移入
归约

	$\langle IDS \rangle$	$\langle IDS \rangle$	$\langle T \rangle$
var	i_A	, i_B	: real

归约-归约冲突

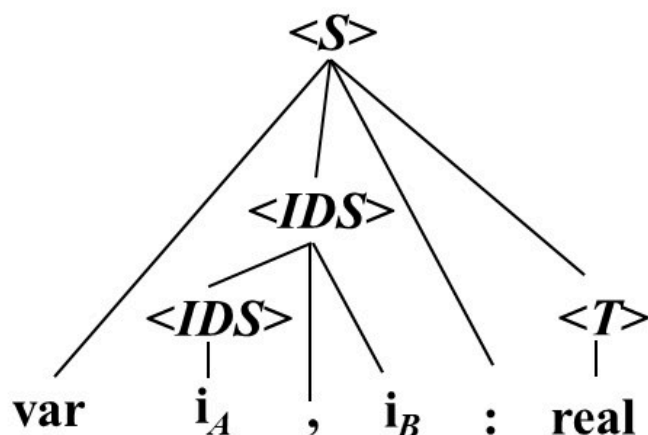
例：

(1) $\langle S \rangle \rightarrow \text{var } \langle IDS \rangle : \langle T \rangle$

(2) $\langle IDS \rangle \rightarrow i$

(3) $\langle IDS \rangle \rightarrow \langle IDS \rangle , i$

(4) $\langle T \rangle \rightarrow \text{real} \mid \text{int}$



造成错误的原因：
错误地识别了句柄

栈

\$
\$ var
\$ var i_A
\$ var <IDS>
\$ var <IDS> ,
\$ var <IDS> , i_B
\$ var <IDS>
\$ var <IDS> :
\$ var <IDS> : real
\$ var <IDS> : <T>
\$ <S>

剩余输入

var i_A, i_B : real \$
i_A, i_B : real \$
, i_B : real \$
, i_B : real \$
i_B : real \$
: real \$
: real \$
real \$
\$
\$
\$

动作

移入
移入
归约
移入
移入
归约
移入
移入
归约
归约

归约-归约冲突

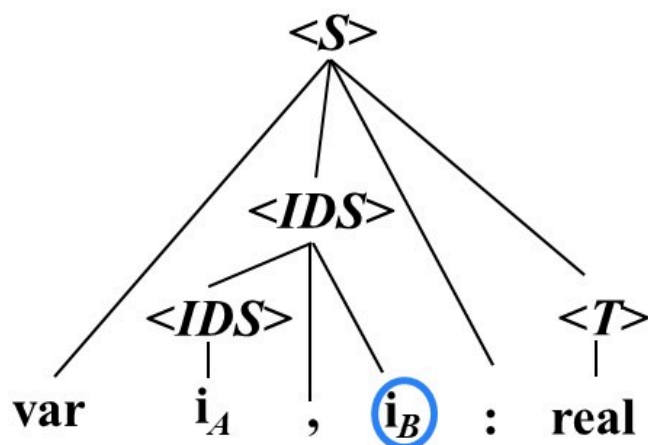
例：

(1) $\langle S \rangle \rightarrow \text{var } \langle IDS \rangle : \langle T \rangle$

(2) $\langle IDS \rangle \rightarrow i$

(3) $\langle IDS \rangle \rightarrow \langle IDS \rangle , i$

(4) $\langle T \rangle \rightarrow \text{real} \mid \text{int}$



造成错误的原因：
错误地识别了句柄

栈

\$
\$ var
\$ var i_A
\$ var <IDS>
\$ var <IDS> ,
\$ var <IDS> , i_B
\$ var <IDS>
\$ var <IDS> :
\$ var <IDS> : real
\$ var <IDS> : <T>
\$ <S>

剩余输入

var i_A, i_B : real \$
i_A, i_B : real \$
, i_B : real \$
, i_B : real \$
i_B : real \$
: real \$
: real \$
real \$
\$
\$
\$

动作

移入
移入
归约
移入
移入
归约
移入
移入
归约
归约

归约-归约冲突

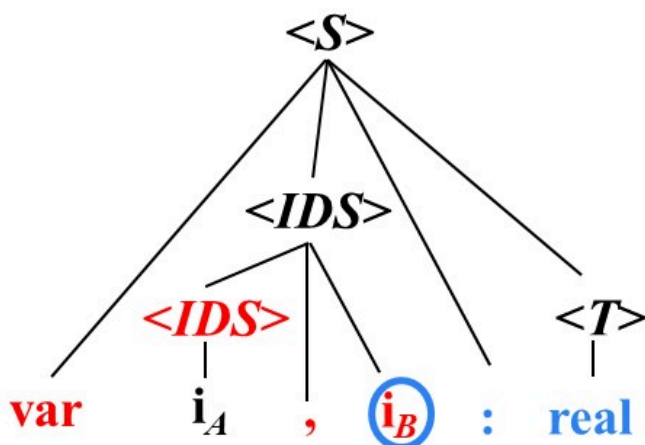
例：

(1) $\langle S \rangle \rightarrow \text{var } \langle IDS \rangle : \langle T \rangle$

(2) $\langle IDS \rangle \rightarrow i$

(3) $\langle IDS \rangle \rightarrow \langle IDS \rangle , i$

(4) $\langle T \rangle \rightarrow \text{real} \mid \text{int}$



造成错误的原因：
错误地识别了句柄

栈

\$
\$ var
\$ var i_A
\$ var <IDS>
\$ var <IDS> ,
\$ var <IDS> , i_B
\$ var <IDS>
\$ var <IDS>
\$ var <IDS>
\$ var <IDS>
\$ var <IDS> : <T>
\$ <S>

剩余输入

var i_A, i_B : real \$
i_A, i_B : real \$
, i_B : real \$
, i_B : real \$
i_B : real \$
: real \$
: real \$

动作

移入

移入

归约

移入

移入

归约

如何正确地识别句柄？

归约

归约

移入-归约冲突

➤ 例

文法:

- (1) $E \rightarrow E+T$
- (2) $E \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow id$

栈

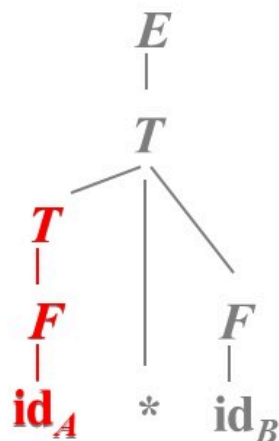
\$
\$ id_A
\$ F
\$ T
\$ E

剩余输入

$id_A * id_B$ \$
\$ $* id_B$ \$
\$ $* id_B$ \$
\$ $* id_B$ \$
\$ $* id_B$ \$

动作

移入
归约
归约
归约





提纲

4.1 自顶向下的分析

4.2 预测分析法

4.3 自底向上的分析

4.4 LR分析法

4.5 语法分析器自动生成工具

4.4 LR 分析法

➤ **LR文法**(*Knuth, 1963*) 是最大的、可以构造出相应
移入-归约语法分析器的文法类

➤ **L**: 对输入进行从**左**到右的扫描

➤ **R**: 反向构造出一个最**右**推导序列

➤ **LR(k)分析**

➤ 在**决定是否归约时**，需要向前查看**k**个输入符号的**LR**
分析

$k = 0$ 和 $k = 1$ 这两种情况具有实践意义
当省略(k)时，表示 $k = 1$



唐纳德·克努斯 (Donald Knuth)
1974年图灵奖得主
TeX排版系统开发者

LR 分析法的基本原理

- 自底向上分析的关键问题是什么？
 - 如何正确地识别句柄
- 句柄是逐步形成的，用“**项目 (item)**”表示句柄识别的进展程度
- 右部某位置标有圆点的产生式称为相应文法的一个**LR(0)项目**（简称为项目）

$$A \rightarrow \alpha_1 \cdot \alpha_2$$

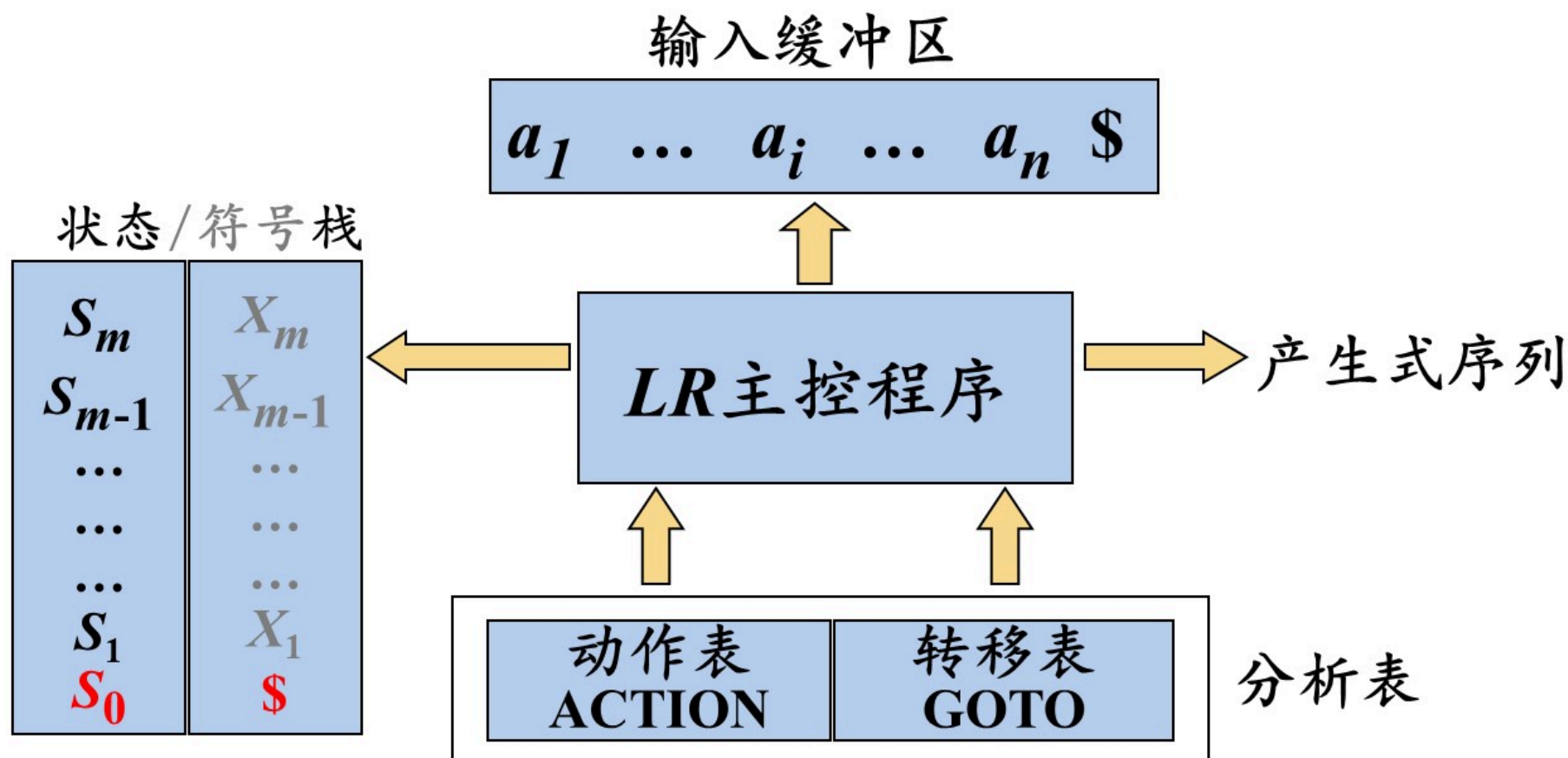
➤ 例： $S \rightarrow bBB$

- $S \rightarrow \cdot bBB$ ← 移进项目
- $S \rightarrow b \cdot BB$
- $S \rightarrow bB \cdot B$ } 待约状态
- $S \rightarrow bBB \cdot$ ← 归约状态

项目描述了句柄识别的状态（进展程度），
LR分析器基于这样一些状态（实际上是由项目组成的集合）来构造自动机进行句柄的识别

产生式 $A \rightarrow \varepsilon$ 只生成一个项目 $A \rightarrow \cdot$

LR 分析器（自动机）的总体结构



LR 分析表的结构

➤ 例

➤ 文法

① $S \rightarrow BB$

② $B \rightarrow aB$

③ $B \rightarrow b$

sn: 将符号 a 、状态 n 压入栈
rn: 用第 n 个产生式进行归约

状态	ACTION			GOTO	
	a	b	$\$$	S	B
0	s3	s4		1	2
1			acc		
2	s3	s4			5
3	s3	s4			6
4	r3	r3	r3		
5	r1	r1	r1		
6	r2	r2	r2		

LR 分析表的结构

➤ 例

➤ 文法

① $S \rightarrow BB$

② $B \rightarrow aB$

③ $B \rightarrow b$

输入 B
 $|$
 $b \ a \ b$

状态	ACTION			GOTO	
	a	b	$\$$	S	B
0	s3	s4		1	2
1			acc		
2	s3	s4			5
3	s3	s4			6
4	r3	r3	r3		
5	r1	r1	r1		
6	r2	r2	r2		

栈

0 4

$\$B$

剩余输入

$bab\$$

LR 分析表的结构

➤ 例

➤ 文法

① $S \rightarrow BB$

② $B \rightarrow aB$

③ $B \rightarrow b$

输入 $\begin{array}{cc} B & B \\ | & | \\ b & a & b \end{array}$

状态	ACTION			GOTO	
	a	b	$\$$	S	B
0	s3	s4		1	2
1			acc		
2	s3	s4			5
3	s3	s4			6
4	r3	r3	r3		
5	r1	r1	r1		
6	r2	r2	r2		

栈
0 2 3 4
\$ B

剩余输入
 $ab \$$

LR 分析表的结构

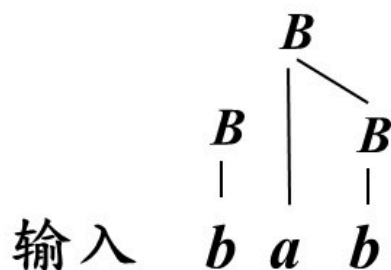
➤ 例

➤ 文法

① $S \rightarrow BB$

② $B \rightarrow aB$

③ $B \rightarrow b$



状态	ACTION			GOTO	
	<i>a</i>	<i>b</i>	\$	<i>S</i>	<i>B</i>
0	s3	s4		1	2
1			acc		
2	s3	s4			5
3	s3	s4			6
4	r3	r3	r3		
5	r1	r1	r1		
6	r2	r2	r2		

栈

0 2 3 6
\$ B a B

剩余输入

\$

LR 分析表的结构

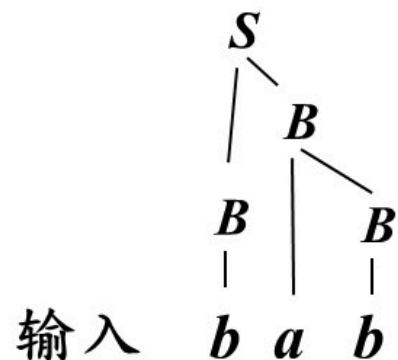
➤ 例

➤ 文法

① $S \rightarrow BB$

② $B \rightarrow aB$

③ $B \rightarrow b$



状态	ACTION			GOTO	
	<i>a</i>	<i>b</i>	\$	<i>S</i>	<i>B</i>
0	s3	s4		1	2
1			acc		
2	s3	s4			5
3	s3	s4			6
4	r3	r3	r3		
5	r1	r1	r1		
6	r2	r2	r2		

栈

0 2 5
\$ B B

剩余输入

\$

LR 分析表的结构

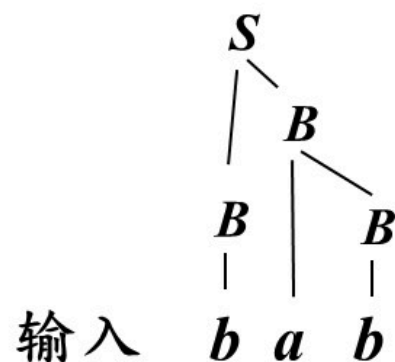
➤ 例

➤ 文法

① $S \rightarrow BB$

② $B \rightarrow aB$

③ $B \rightarrow b$



状态	ACTION			GOTO	
	<i>a</i>	<i>b</i>	\$	<i>S</i>	<i>B</i>
0	s3	s4		1	2
1			acc		
2	s3	s4			5
3	s3	s4			6
4	r3	r3	r3		
5	r1	r1	r1		
6	r2	r2	r2		

栈

0 1
\$ S

剩余输入

\$

LR 分析表的结构

➤ 例

➤ 文法

① $S \rightarrow BB$

② $B \rightarrow aB$

③ $B \rightarrow b$

状态	ACTION			GOTO	
	a	b	$\$$	S	B
0	s3	s4		1	2
1			acc		
2	s3	s4			5
3	s3	s4			6
4	r3	r3	r3		
5	r1	r1	r1		
6	r2	r2	r2		

$$s_i \xrightarrow[A(\text{归约})]{a(\text{移入})} s_j$$

LR 分析器的工作过程

➤ 初始化

s_0	
$\$$	$a_1 a_2 \dots a_n \$$

➤ 一般情况下

$s_0 s_1 \dots s_m$	
$\$X_1 \dots X_m$	$a_i a_{i+1} \dots a_n \$$

① 如果 ACTION $[s_m, a_i] = sx$, 那么格局变为:

$s_0 s_1 \dots s_m x$	
$\$X_1 \dots X_m a_i$	$a_{i+1} \dots a_n \$$

LR 分析器的工作过程

➤ 初始化

s_0	
\$	$a_1 a_2 \dots a_n$ \$

➤ 一般情况下

$s_0 s_1 \dots s_m$	
\$ $X_1 \dots X_m$	$a_i a_{i+1} \dots a_n$ \$

② 如果 $\text{ACTION}[s_m, a_i] = rx$ 表示用第 x 个产生式 $A \rightarrow X_{m-(k-1)} \dots X_m$

进行归约，那么格局变为：

$s_0 s_1 \dots s_{m-k}$	
\$ $X_1 \dots X_{m-k} A$	$a_i a_{i+1} \dots a_n$ \$

如果 $\text{GOTO}[s_{m-k}, A] = y$ ，那么格局变为：

$s_0 s_1 \dots s_{m-k} y$	
\$ $X_1 \dots X_{m-k} A$	$a_i a_{i+1} \dots a_n$ \$

LR 分析器的工作过程

➤ 初始化

s_0 $\$$	$a_1 a_2 \dots a_n \$$
---------------	------------------------

➤ 一般情况下

$s_0 s_1 \dots s_m$ $\$X_1 \dots X_m$	$a_i a_{i+1} \dots a_n \$$
--	----------------------------

③ 如果 $\text{ACTION}[s_m, a_i] = acc$, 那么分析成功

④ 如果 $\text{ACTION}[s_m, a_i] = err$, 那么出现语法错误

LR 分析算法

- 输入：串 w 和LR语法分析表，该表描述了文法 G 的ACTION函数和GOTO函数。
- 输出：如果 w 在 $L(G)$ 中，则输出 w 的自底向上语法分析过程中的归约步骤；否则给出一个错误指示。
- 方法：初始时，语法分析器栈中的内容为初始状态 s_0 ，输入缓冲区中的内容为 $w\$$ 。然后，语法分析器执行下面的程序：

```
令 $a$ 为 $w\$$ 的第一个符号；
while(1) { /* 永远重复*/
    令 $s$ 是栈顶的状态；
    if ( ACTION [ $s, a$ ] = st ) {
        将 $t$ 压入栈中；
        令 $a$ 为下一个输入符号；
    } else if ( ACTION [ $s, a$ ] = 归约  $A \rightarrow \beta$  ) {
        从栈中弹出  $|\beta|$  个符号；
        将GOTO [ $t, A$ ]压入栈中；
        输出产生式  $A \rightarrow \beta$ ；
    } else if ( ACTION [ $s, a$ ] = 接受 ) break; /* 语法分析完成*/
    else 调用错误恢复例程；
}
```

如何构造给定文法的 LR 分析表?

- $LR(0)$ 分析
- SLR 分析
- $LR(1)$ 分析
- $LALR$ 分析

4.4.1 LR(0) 分析

➤ 增广文法 (Augmented Grammar)

➤ 如果 G 是一个以 S 为开始符号的文法，则 G 的增广文法 G' 就是在 G 中加上新开始符号 S' 和产生式 $S' \rightarrow S$ 而得到的文法

➤ 例

1) $E \rightarrow E + T$
2) $E \rightarrow T$
3) $T \rightarrow T * F$
4) $T \rightarrow F$
5) $F \rightarrow (E)$
6) $F \rightarrow \text{id}$

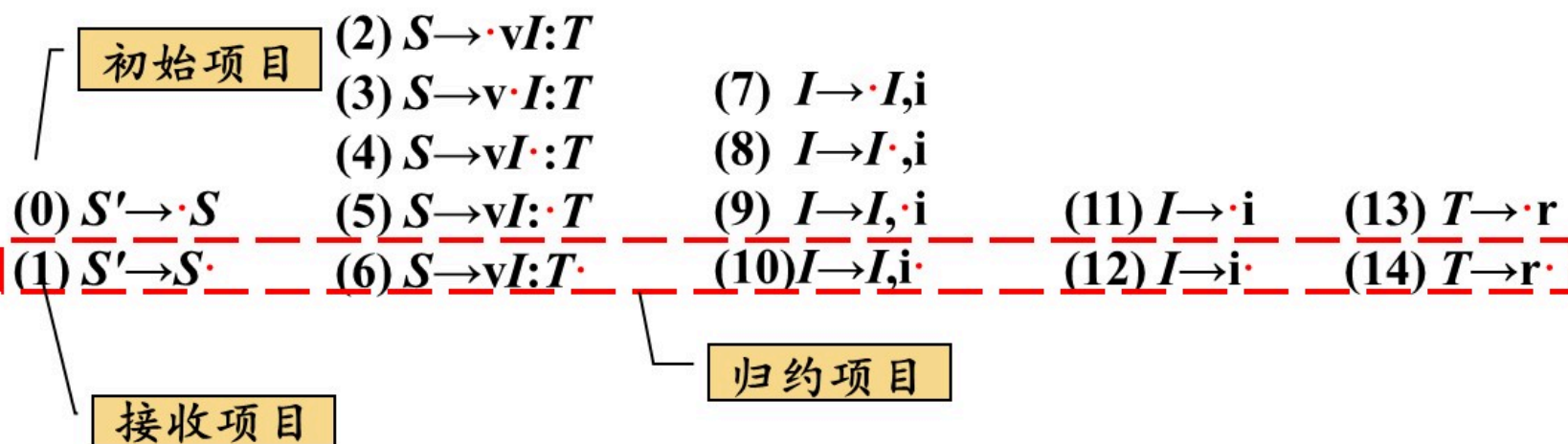


0) $E' \rightarrow E$
1) $E \rightarrow E + T$
2) $E \rightarrow T$
3) $T \rightarrow T * F$
4) $T \rightarrow F$
5) $F \rightarrow (E)$
6) $F \rightarrow \text{id}$

引入这个新的开始产生式的目的是使得文法开始符号仅出现在一个产生式的左边，从而使得分析器只有一个接收状态

例：LR(0)自动机的构造

① $S' \rightarrow S$ ② $S \rightarrow vI:T$ ③ $I \rightarrow I,i$ ④ $I \rightarrow i$ ⑤ $T \rightarrow r$



例：LR(0)自动机的构造

① $S' \rightarrow S$ ② $S \rightarrow vI:T$ ③ $I \rightarrow I,i$ ④ $I \rightarrow i$ ⑤ $T \rightarrow r$

	(2) $S \rightarrow \cdot vI:T$			
	(3) $S \rightarrow v \cdot I:T$	(7) $I \rightarrow \cdot I,i$		
	(4) $S \rightarrow vI \cdot :T$	(8) $I \rightarrow I \cdot ,i$		
(0) $S' \rightarrow \cdot S$	(5) $S \rightarrow vI: \cdot T$	(9) $I \rightarrow I, \cdot i$	(11) $I \rightarrow \cdot i$	(13) $T \rightarrow \cdot r$
(1) $S' \rightarrow S \cdot$	(6) $S \rightarrow vI:T \cdot$	(10) $I \rightarrow I,i \cdot$	(12) $I \rightarrow i \cdot$	(14) $T \rightarrow r \cdot$

➤ 后继项目 (Successive Item)

➤ 同属于一个产生式的项目，但圆点的位置只相差一个符号，
则称后者是前者的后继项目

➤ $A \rightarrow \alpha \cdot X\beta$ 的后继项目是 $A \rightarrow \alpha X \cdot \beta$

例：LR(0)自动机的构造

① $S' \rightarrow S$ ② $S \rightarrow vI:T$ ③ $I \rightarrow I,i$ ④ $I \rightarrow i$ ⑤ $T \rightarrow r$

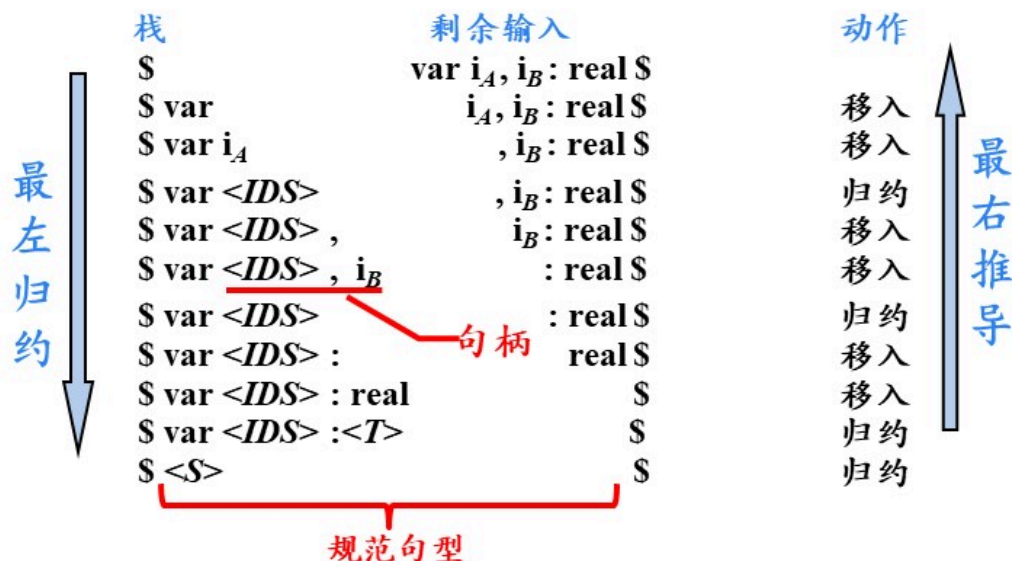
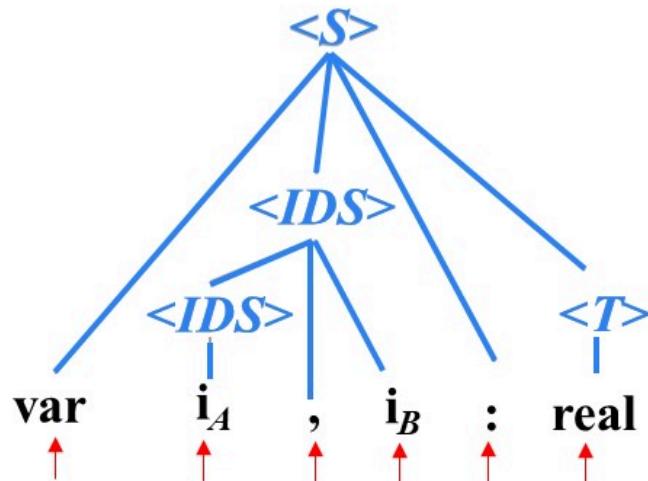
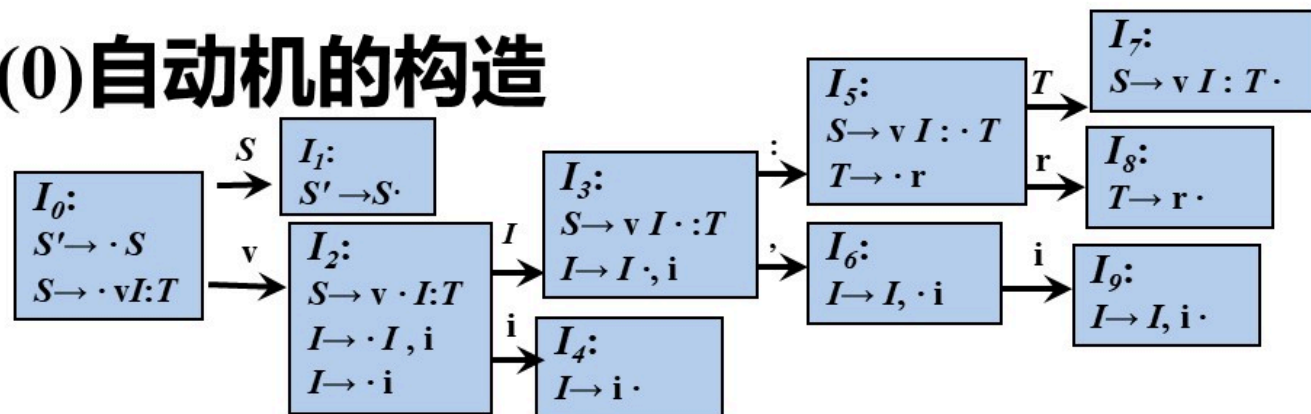
	(2) $S \rightarrow \cdot vI:T$			
	(3) $S \rightarrow v \cdot I:T$	(7) $I \rightarrow \cdot I,i$		
	(4) $S \rightarrow vI \cdot :T$	(8) $I \rightarrow I \cdot ,i$		
(0) $S' \rightarrow \cdot S$	(5) $S \rightarrow vI \cdot :T$	(9) $I \rightarrow I, \cdot i$	(11) $I \rightarrow \cdot i$	(13) $T \rightarrow \cdot r$
(1) $S' \rightarrow S \cdot$	(6) $S \rightarrow vI:T \cdot$	(10) $I \rightarrow I,i \cdot$	(12) $I \rightarrow i \cdot$	(14) $T \rightarrow r \cdot$

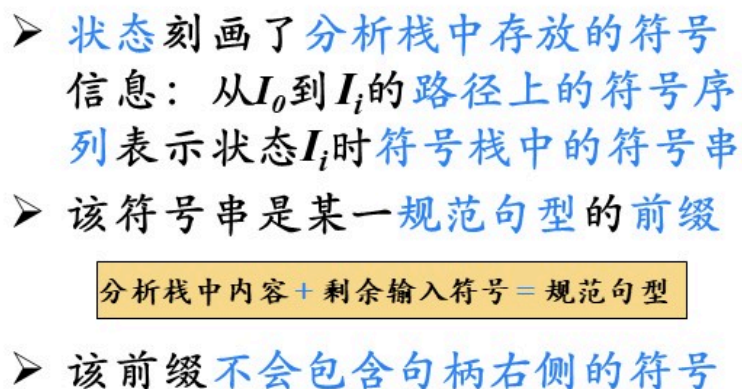
这15个项目中是否会有某些项目是等价的？

可以把等价的项目组成一个项目集(I)，称为**项目集闭包** (Closure of Item Sets)，每个项目集闭包对应着**自动机**的一个**状态**

例：LR(0)自动机的构造

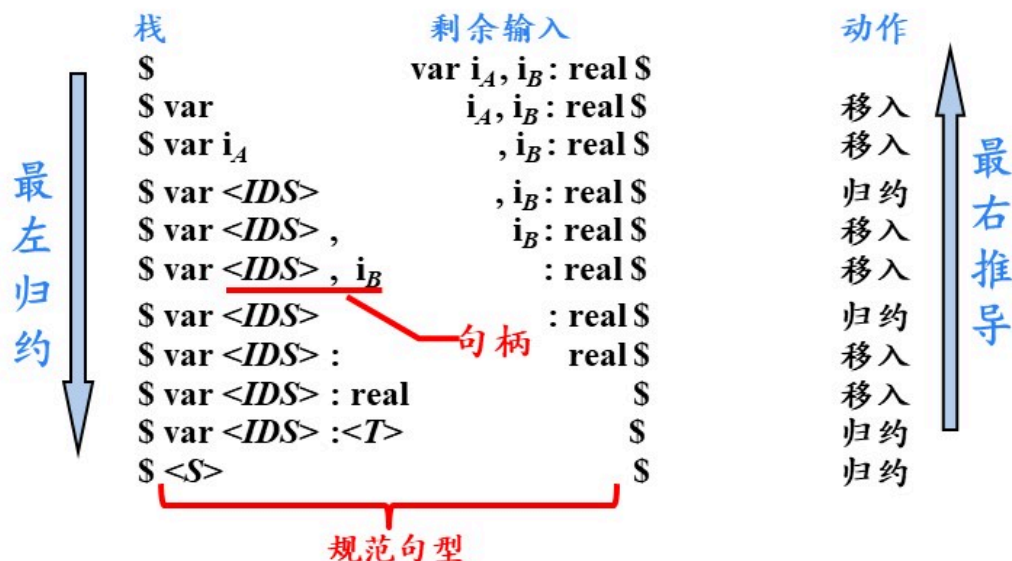
- ① $S' \rightarrow S$
- ② $S \rightarrow vI:T$
- ③ $I \rightarrow I, i$
- ④ $I \rightarrow i$
- ⑤ $T \rightarrow r$





分析栈中内容 + 剩余输入符号 = 规范句型

➤ 该前缀不会包含句柄右侧的符号



例：LR(0)自动机的构造

规范句型的**活前缀** (Active Prefix)：

不含**句柄右侧**任意符号的**规范句型的前缀**

- 状态刻画了分析栈中存放的符号信息：从 I_0 到 I_i 的路径上的符号序列表示状态 I_i 时符号栈中的符号串

- 该符号串是某一规范句型的前缀

分析栈中内容 + 剩余输入符号 = 规范句型

- 该前缀**不会包含句柄右侧**的符号

最左归约

栈	剩余输入
\$	var i_A, i_B : real S
\$ var	i_A, i_B : real S
\$ var i_A	, i_B : real S
\$ var <IDS>	, i_B : real S
\$ var <IDS> ,	i_B : real S
\$ var <IDS> , i_B	: real S
\$ var <IDS> : real S	: real S
\$ var <IDS> :	real S
\$ var <IDS> : real	\$
\$ var <IDS> :<T>	\$
\$ <S>	\$

句柄

规范句型

动作

移入
移入
归约
移入
移入
归约
移入
移入
归约
归约

最右推导

例：LR(0)自动机的构造

规范句型的**活前缀** (Active Prefix) :
不含句柄右侧任意符号的规范句型的前缀

规范句型 $\text{var } \langle \text{IDS} \rangle, i : \text{real}$ 的前缀

- var
 - var <IDS>
 - var <IDS> ,
 - var <IDS> , i
 - var <IDS> , i :
 - var <IDS> , i : real
- 不会出现在分析栈中

- 状态刻画了分析栈中存放的符号信息：从 I_0 到 I_i 的路径上的符号序列表示状态 I_i 时符号栈中的符号串
 - 该符号串是某一规范句型的前缀
- 分析栈中内容 + 剩余输入符号 = 规范句型
- 该前缀不会包含句柄右侧的符号

最左归约

栈	剩余输入
\$	var $i_A, i_B : \text{real}$ S
\$ var	$i_A, i_B : \text{real}$ S
\$ var i_A	, $i_B : \text{real}$ S
\$ var <IDS>	, $i_B : \text{real}$ S
\$ var <IDS> ,	$i_B : \text{real}$ S
\$ var <IDS> , i_B	: real S
\$ var <IDS> : real S	: real S
\$ var <IDS> : real S	real S
\$ var <IDS> : real S	S
\$ var <IDS> : <T>	S
\$ <S>	S

句柄

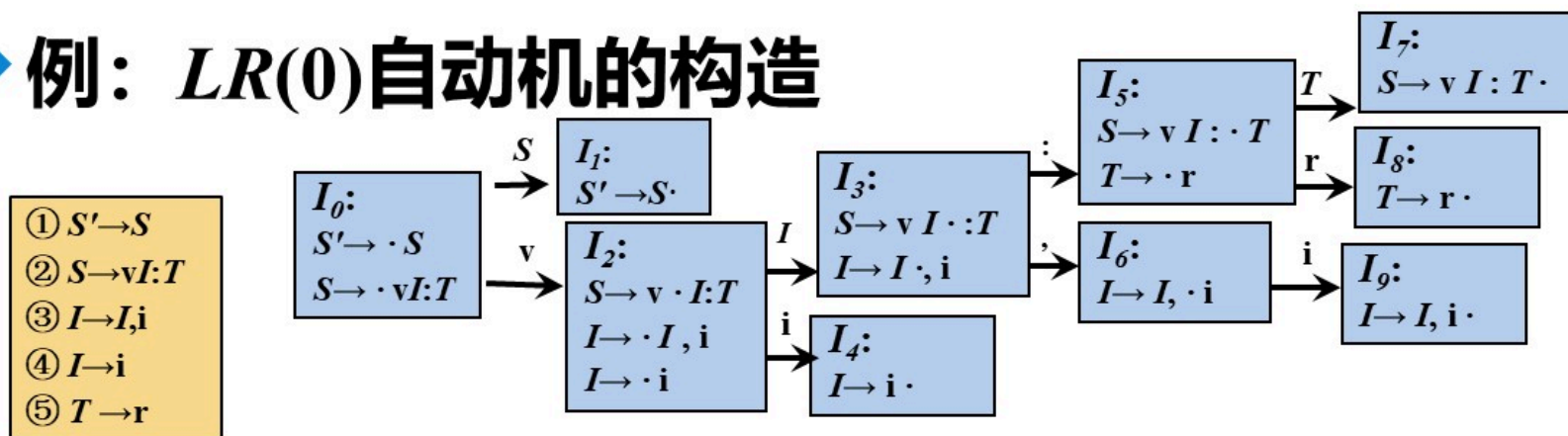
规范句型

动作

移入
移入
归约
移入
移入
归约
移入
移入
归约
归约

最右推导

例：LR(0)自动机的构造



➤ 程序实现

- **CLOSURE(I)**函数：计算给定项目集 I 的闭包
- **GOTO(I, X)**函数：返回项目集 I 对应于文法符号 X 的**后继项目集闭包**

CLOSURE()函数

➤ 计算给定项目集 I 的闭包

$$\text{CLOSURE}(I) = I \cup \{B \rightarrow \cdot \gamma \mid A \rightarrow \alpha \cdot B \beta \in \text{CLOSURE}(I), B \rightarrow \gamma \in P\}$$

```
SetOfItems CLOSURE ( I ) {  
    J = I;  
    repeat  
        for ( J中的每个项  $A \rightarrow \alpha \cdot B \beta$  )  
            for ( G的每个产生式  $B \rightarrow \gamma$  )  
                if ( 项  $B \rightarrow \cdot \gamma$  不在 J 中 )  
                    将  $B \rightarrow \cdot \gamma$  加入 J 中;  
    until 在某一轮中没有新的项被加入到 J 中;  
    return J;  
}
```


GOTO ()函数

➤ 返回项目集 I 对应于文法符号 X 的 **后继** 项目集闭包

$$\text{GOTO}(I, X) = \text{CLOSURE}(\{A \rightarrow \alpha X \cdot \beta \mid A \rightarrow \alpha \cdot X \beta \in I\})$$

```
SetOfItems GOTO ( I, X ) {  
    将  $J$  初始化为空集;  
    for (  $I$  中的每个项  $A \rightarrow \alpha \cdot X \beta$  )  
        将项  $A \rightarrow \alpha X \cdot \beta$  加入到集合  $J$  中;  
    return CLOSURE (  $J$  );  
}
```

构造 $LR(0)$ 自动机的状态集

➤ 规范 $LR(0)$ 项集族(*Canonical $LR(0)$ Collection*)

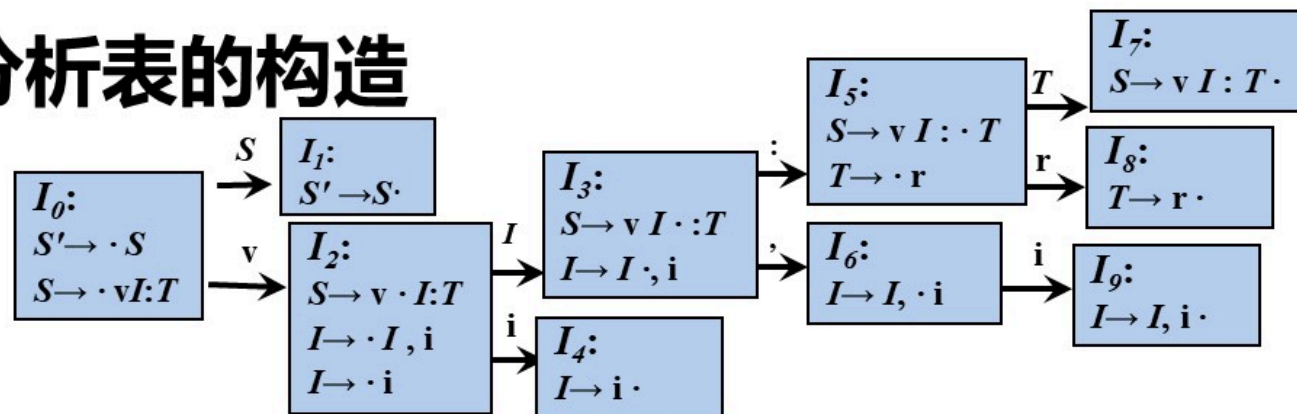
$$C = \{I_0\} \cup \{I \mid \exists J \in C, X \in V_N \cup V_T, I = GOTO(J, X)\}$$

```
void items(  $G'$  ) {  
     $C = \{ \text{CLOSURE}(\{[S' \rightarrow \cdot S]\}) \};$   
    repeat  
        for ( $C$ 中的每个项集  $I$ )  
            for(每个文法符号  $X$ )  
                if (  $GOTO(I, X)$  非空且不在  $C$  中)  
                    将  $GOTO(I, X)$  加入  $C$  中;  
    until 在某一轮中没有新的项集被加入到  $C$  中;  
}
```

LR(0)分析表的构造

➤ 例

- ① $S' \rightarrow S$
- ② $S \rightarrow vI:T$
- ③ $I \rightarrow I,i$
- ④ $I \rightarrow i$
- ⑤ $T \rightarrow r$



状态	ACTION						GOTO		
	v	:	,	i	r	\$	S	I	T
0	s2						1		
1						acc			
2				s4				3	
3		s5	s6						
4	r4	r4	r4	r4	r4	r4			
5					s8				7
6				s9					
7	r2	r2	r2	r2	r2	r2			
8	r5	r5	r5	r5	r5	r5			
9	r3	r3	r3	r3	r3	r3			

信息为空的项目
都设置为“error”

主观题 2分

1. 自顶向下语法分析的通用形式是什么?
2. 自底向上语法分析的通用形式是什么?

单选题 2分

在自底向上的语法分析方法中，分析的关键是()。

- ☐ A 选择候选式
- ☐ B 消除递归
- ☐ C 寻找句柄
- ☐ D 寻找句型

➤ 上一讲内容回顾（自底向上分析）

- 自底向上语法分析的通用框架
 - 移入-归约分析(*Shift-Reduce Parsing*)
- 移入-归约分析中存在的问题
 - 归约-归约冲突
 - 移入-归约冲突
- 什么样的文法不存在以上两类冲突，从而可以做**确定的**自底向上分析？
 - LR文法
 - LR(0)、SLR、LR(1)、LALR

上一讲内容回顾（自底向上分析）

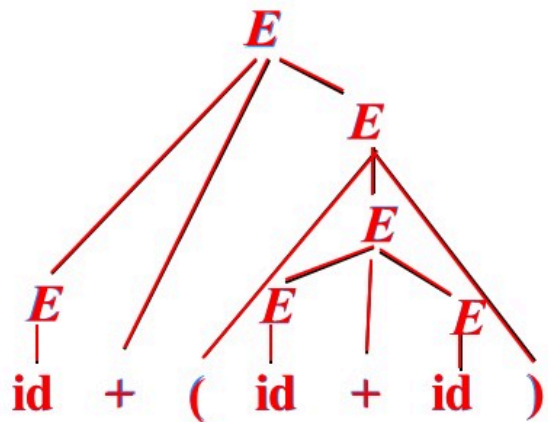
- 自底向上语法分析的通用框架
 - 移入-归约分析(*Shift-Reduce Parsing*)

例：移入-归约分析

栈内符号串 + 剩余输入 = “规范句型”

文法

- ① $E \rightarrow E+E$
- ② $E \rightarrow E * E$
- ③ $E \rightarrow (E)$
- ④ $E \rightarrow id$



最左归约

栈	剩余输入	动作
\$	id+(id+id) \$	
\$ id	+(id+id) \$	移入
\$ E	+(id+id) \$	归约: $E \rightarrow id$
\$ E +	(id+id) \$	移入
\$ E + (	id+id) \$	移入
\$ E + (id	+id) \$	移入
\$ E + (E	+id) \$	归约: $E \rightarrow id$
\$ E + (E +	id) \$	移入
\$ E + (E + id	) \$	移入
\$ E + (E + E	) \$	归约: $E \rightarrow id$
\$ E + (E	) \$	归约: $E \rightarrow E + E$
\$ E + (E)	\$	移入
\$ E + E	\$	归约: $E \rightarrow (E)$
\$ E	\$	归约: $E \rightarrow E + E$

最右推导

➤ 上一讲内容回顾（自底向上分析）

- 自底向上语法分析的通用框架
 - 移入-归约分析(*Shift-Reduce Parsing*)
- 移入-归约分析中存在的问题
 - 归约-归约冲突
 - 移入-归约冲突
- 什么样的文法不存在以上两类冲突，从而可以做**确定的**自底向上分析？
 - LR文法
 - LR(0)、SLR、LR(1)、LALR

LR(0)分析表构造方法

➤ $CLOSURE(\{[S' \rightarrow \cdot S]\}) \rightarrow C$

C: 自动机状态集合

➤ for each $I_i \in C$

➤ if $A \rightarrow \alpha \cdot a \beta \in I_i$: 令 $I_j = GOTO(I_i, a)$; $ACTION[i, a] = s_j$

➤ if $A \rightarrow \alpha \cdot B \beta \in I_i$: 令 $I_j = GOTO(I_i, B)$; $GOTO[i, B] = j$

➤ if $A \rightarrow \alpha \cdot \in I_i$: $\begin{cases} \text{if } A = S' \text{ (即 } S' \rightarrow S \cdot \in I_i) & ACTION[i, \$] = acc \\ \text{if } A \neq S' & \text{for } \forall a \in V_T \cup \{\$\} \text{ do } ACTION[i, a] = r_j \text{ (} j \text{ 是产生式 } A \rightarrow a \text{ 的编号)} \end{cases}$

➤ 没有定义的所有条目都设置为“error”

LR(0) 自动机的形式化定义

➤ 文法

$$G = (V_N, V_T, P, S)$$

➤ LR(0) 自动机

$$M = (C, V_N \cup V_T, GOTO, I_0, F)$$

$$➤ C = \{I_0\} \cup \{I \mid \exists J \in C, X \in V_N \cup V_T, I = GOTO(J, X)\}$$

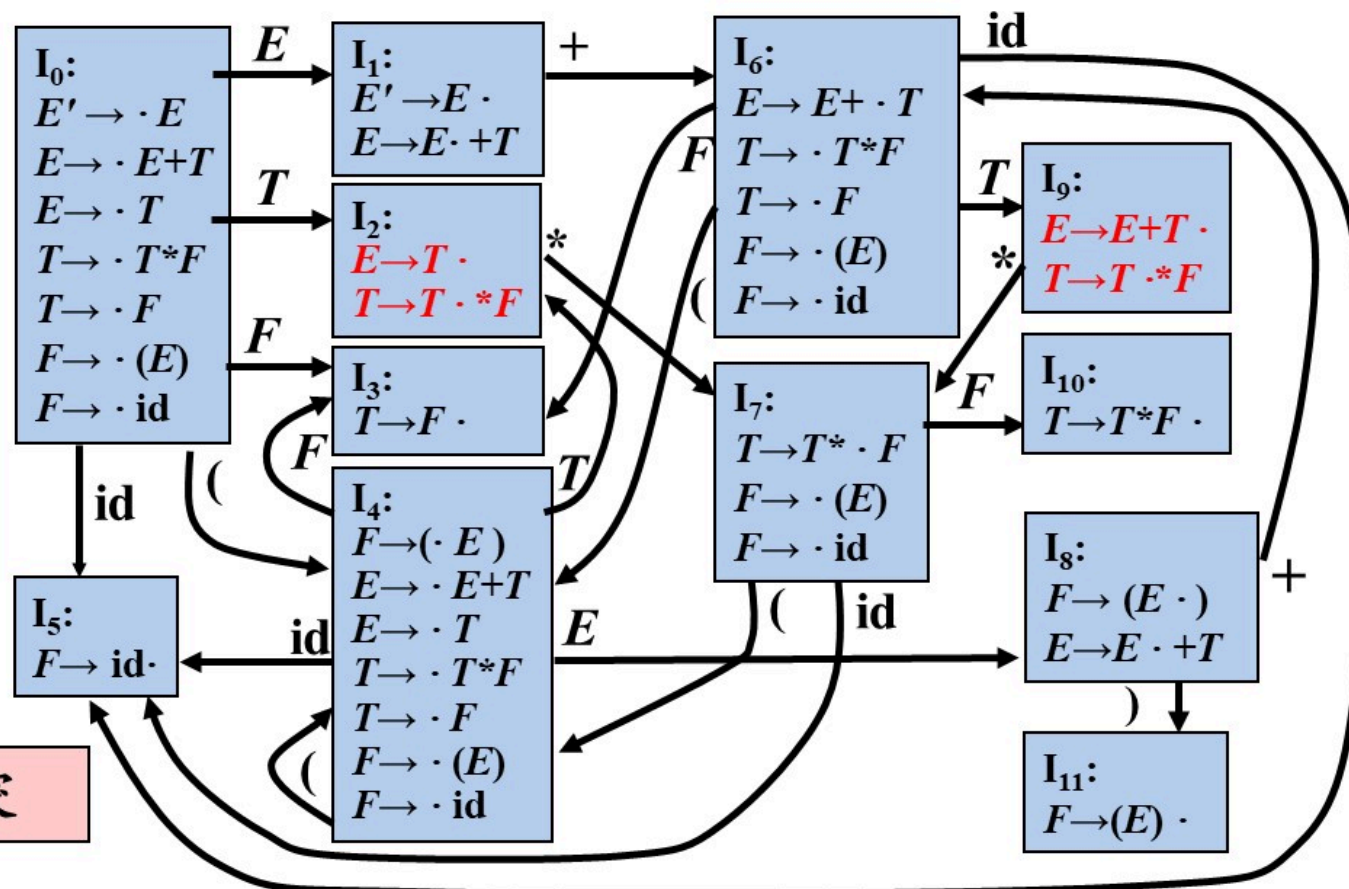
$$➤ I_0 = CLOSURE(\{S' \rightarrow \cdot S\})$$

LR(0) 分析过程中的冲突

文法

- (0) $E' \rightarrow E$
- (1) $E \rightarrow E+T$
- (2) $E \rightarrow T$
- (3) $T \rightarrow T*F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow id$

移进/归约冲突



表达式文法的 $LR(0)$ 分析表含有移进/归约冲突

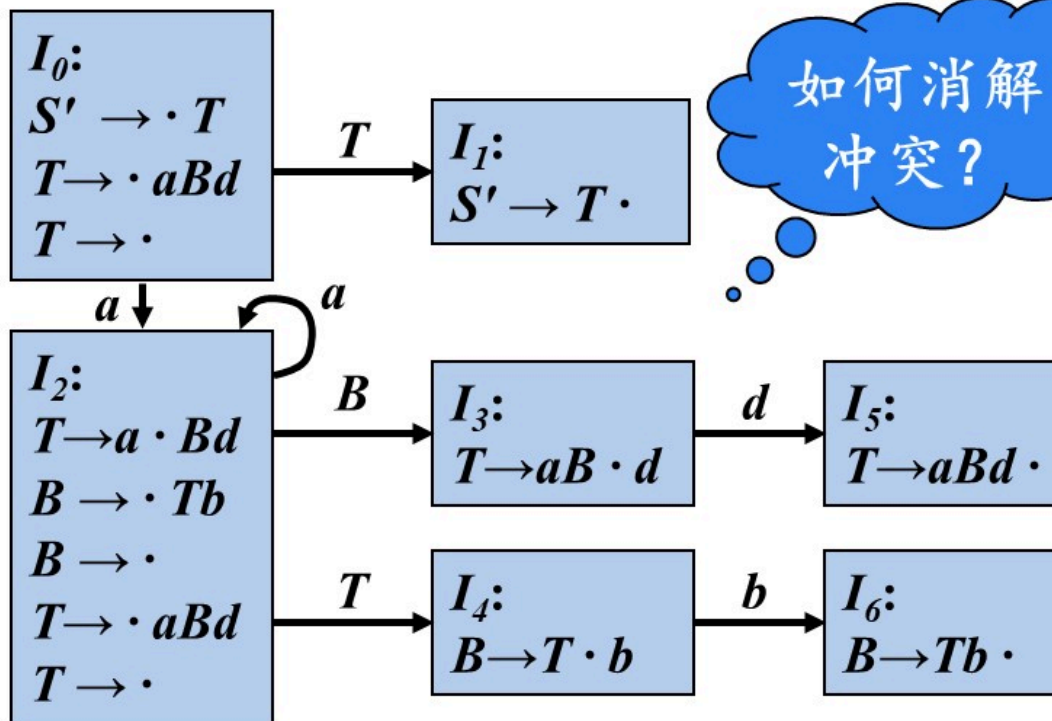
状态	ACTION						GOTO		
	id	+	*	(	)	\$	E	T	F
0	s5			s ₄			1	2	3
1		s6				acc			
2	r2	r2	r2/s7	r2	r2	r2			
3	r4	r4	r4	r4	r4	r4			
4	s5			s4			8	2	3
5	r6	r6	r6	r6	r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9	r1	r1	r1/s7	r1	r1	r1			
10	r3	r3	r3	r3	r3	r3			
11	r5	r5	r5	r5	r5	r5			

例：移进/归约冲突和归约/归约冲突

文法

- (0) $S' \rightarrow T$
- (1) $T \rightarrow aBd$
- (2) $T \rightarrow \varepsilon$
- (3) $B \rightarrow Tb$
- (4) $B \rightarrow \varepsilon$

如果LR(0)分析表中
没有语法分析动作冲突，那么给定的文法
就称为LR(0)文法



不是所有CFG都能用LR(0)方法进行分析，也就是说，CFG不总是LR(0)文法